THREE SIGMA

[un]wind

Security Review

Disclaimer

Security Review

[un]wind

Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

[un]wind

Table of Contents

Summary

Security Review

[un]wind

Summary

Three Sigma audited [un]wind in a 0.8 person week engagement. The audit was conducted from 27/07/2026 to 28/07/2026.

Protocol Description

[un]windis a protocol that manages the full lifecycle of leveraged positions on tokenized Real-World Assets (RWAs) within Euler V2 lending markets. The system consists of two core modules, Wind and Unwind, that enable users to atomically enter and exit leveraged RWA positions despite the asynchronous settlement nature of the underlying assets. During a Wind, the protocol deploys per-request escrow contracts that mint a temporary interim collateral token (PSBA), borrow from Euler, and queue deposits into Pareto credit vaults (Idle Finance); once the RWA shares are claimable, an operator settles the queue and finalizes each position by swapping the interim collateral for real RWA shares and transferring the CDP to the user. During an Unwind, the reverse process locks the user's CDP, replaces RWA collateral with PSBA, queues a redemption through Pareto, and upon settlement returns the underlying proceeds minus fees. The protocol integrates with the Ethereum Vault Connector (EVC) for batched vault operations, Pyth Network for price feed validation, and Keyring Network for KYC gating, and relies on role-gated operators to coordinate queue settlement and liquidation of unhealthy positions.

Scope

Security Review

[un]wind

Scope

Filepath	nSLOC
rwa-unwind/contracts/src/unwind/euler/adapters/MidasAdapter.sol	224
euler-keyring-hooks/src/hooks/HookTargetAccessControlKeyringUnwind.sol	50
Total	**274**

Methodology

Security Review

[un]wind

To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

[un]wind

Project Dashboard

Application Summary

Name	[un]wind
Repository	https://github.com/Keyring-Network/rwa-unwind https://github.com/Keyring-Network/euler-keyring-hooks
Commit	Branch audit/round-3
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	27/07/2026 to 28/07/2026
№ of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	2	0	2
Medium	2	0	2
Low	0	0	0
None	2	0	2

Category Breakdown

Suggestion	1
Documentation	0
Bug	0
Optimization	1
Good Code Practices	0

Risk Section

Security Review

[un]wind

Risk Section

No risk was identified.

Findings
Security Review
[un]wind

Findings

3S-M01

safetyCloseUnwind zeroes the payout pool it unlocks, leading to total loss of an auction winner's deposited capital

Id	3S--M01
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged with note: “safetyCloseUnwind() is a privileged, admin-only function invoked when the underlying fund defaults, the user is sanctioned, or other uneventful situations. In this scenario, the protocol team (Keyring) covers the outstanding debt from its treasury (or via an insurance claim), with the sole purpose of preventing bad debt from accruing to the cluster. Liquidator payouts are intentionally not honored through this route; where applicable, they may instead be settled off-chain.”

Description

When an unwind escrow becomes liquidatable, the protocol runs a liquidation auction. The winning bidder repays the escrow's debt out of their own pocket up front, and their compensation (**actualRepay + bestBonus**) is deferred: they claim it later through **UnwindLiquidationAuction::claimWinningRoundPayout**, which pays out of a per-request pool, **s_requestIdToClaimableAssets**. That pool is normally filled by **finalizeUnwind** once the redemption settles.

The winner cannot claim until the request is finalized. Two transitions finalize a request. The normal one is **finalizeUnwind**, which fills the pool. The other is **UnwindManagerExtension::safetyCloseUnwind**, the emergency close for a request that is stuck — for example when a Midas redemption is stuck. **safetyCloseUnwind** marks the

escrow **Finalized**, unlocking the winner's claim, but it never funds the reserved payout — it leaves the pool at zero (only **finalizeUnwind** ever fills it) and never reconciles the winner's **s_unpaidReservedPayout**.

So after a safety close the winner's claim is unlocked, but the pool it draws from is empty. The payout is computed pro-rata against that pool:

```
// UnwindManagerExtension.sol:387-388
uint256 remainingPayoutPool = s_requestIdToClaimableAssets[_requestId];         // =
0 (never funded; only finalizeUnwind fills it)
uint256 payoutAmount = (remainingPayoutPool * _amount) /
unpaidLiquidationPayout; // = (0 * x) / x = 0
```

The winner receives nothing, yet the round is marked **Claimed** irreversibly (the status is set before the transfer, **UnwindLiquidationAuction.sol:302**), and their bid bond was already zeroed at execution — so there is no path back. Their capital, which already repaid the escrow's debt and thereby reduced the amount the safety manager had to top up, effectively subsidizes the close. All of this happens with no revert and no warning event.

Steps to Reproduce

1. A user unwinds a position; the escrow enters **Redeeming** with no settled assets (**s_requestIdToClaimableAssets == 0**).

2. Price movement or accrued interest makes the escrow liquidatable; a round opens and a winner calls **executeWinningRound**, repaying the debt from their own funds. Their payout is reserved but deferred, and their bond is zeroed.

3. The redemption never settles (for example, Midas rejects it), so the pool stays at zero.

4. The safety manager calls **safetyCloseUnwind**: it tops up the outstanding debt and marks the escrow **Finalized**, but never funds or reconciles the reserved payout, so the pool stays at zero.

5. The winner calls **claimWinningRoundPayout**: **isRequestFinalized** is now true, but the payout computes to **(0 * amount) / unpaid = 0**. They receive nothing and the round is marked **Claimed** permanently.

Recommendation

Have **safetyCloseUnwind** also check **getUnpaidReservedPayout** and, when it is non-zero, fund that amount from the safety manager and fill **s_requestIdToClaimableAssets** with it instead of leaving it at zero, so the existing claim

path pays the winner in full. The only added cost is the auction **bonus**, which the protocol absorbs since no redemption proceeds exist to fund it.

3S-M02

Burning the receipt NFT in **safetyCloseUnwind** without closing the adapter request leads to a permanently reverting settlement and stranded redemption proceeds

Id	3S--M02
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged with note: “safetyCloseUnwind() is a privileged, admin-only function invoked when the underlying fund defaults, the user is sanctioned, or other uneventful situations. In this scenario, the protocol team (Keyring) covers the outstanding debt from its treasury (or via an insurance claim), with the sole purpose of preventing bad debt from accruing to the cluster. Liquidator payouts are intentionally not honored through this route; where applicable, they may instead be settled off-chain.”

Description

UnwindManagerExtension::safetyCloseUnwind is the operator's escape hatch for an unwind request that the redemption provider has not processed. It is only callable while nothing has been settled, which the guard at **UnwindManagerExtension.sol:234-237** enforces:

```
        uint256 claimableAssets = s_requestIdToClaimableAssets[_requestId];
        if (claimableAssets != 0) {
            revert UnwindManagerExtension__SafetyCloseUnavailable(_requestId,
claimableAssets);
        }
```

The function closes the escrow, has **SAFETY_MANAGER_ROLE** repay the outstanding debt from its own balance, and burns the user's receipt NFT at **UnwindManagerExtension.sol:259**. It never notifies the adapter. The adapter request remains **Requested** and the Midas request remains **Pending**, so nothing has actually been cancelled on either side. The position has simply been abandoned by the manager.

When Midas later approves that request, the payment asset is delivered to the user wallet as normal. A keeper then calls **settleRequest**. The claimable gate passes, **MidasAdapter::claimRedeem** succeeds and moves the assets to the manager, and the transaction reaches the final line of **UnwindManager::_settleRequestIfClaimable** at **UnwindManager.sol:468-475**:

```
        s_requestIdToClaimableAssets[_requestId] += redeemedAssets;

EnumerableSet.remove(s_pendingRequestIdsByUser[ownerOf(requestState.receiptNftId)], _requestId);
```

ownerOf resolves through **_requireOwned** in OpenZeppelin 5.4.0, which reverts on a burned id rather than returning the zero address:

```
function _requireOwned(uint256 tokenId) internal view returns (address) {
    address owner = _ownerOf(tokenId);
    if (owner == address(0)) {
        revert ERC721NonexistentToken(tokenId);
    }
    return owner;
}
```

The whole transaction reverts, including the successful claim. Nothing about that state ever changes, so the call fails identically on every retry.

The proceeds are then stranded in the user wallet. **claimRedeem** is the only path that moves the payment asset out of the wallet, and it is reachable only through **_settleRequestIfClaimable**, which always reverts at the line above. The wallet gate in **UserWallet::batchCall** is looser than the bound adapter alone:

```
        return IWindManagerFactory(windManagerFactory).isManagerActive(_manager)
            ||
IUnwindManagerFactory(unwindManagerFactory).isManagerActive(_manager);
```

Any active wind or unwind manager's adapter is accepted, not only the bound one, so the recovery surface is wider than a single contract. No shipped adapter exposes a sweep, so the funds remain stuck in practice.

This is not a misuse of the function. It is documented as closing a stuck request, and stuck normally means the provider is slow rather than that the provider rejected. Midas skips requests silently when the redeemer is short on liquidity, inside the bulk approval loop of **RedemptionVault::safeBulkApproveRequest**:

```
if (!success) {
    continue;
}
```

No event is emitted on that path, so an operator has no on-chain signal distinguishing a request that is merely queued from one that will never be approved.

Steps to Reproduce

1. A user unwind stalls. Midas has the request at **Pending** and **safeBulkApproveRequest** has skipped it twice for short redeemer liquidity, emitting nothing

2. The operator cannot distinguish this from a request Midas will never approve, and calls **safetyCloseUnwind**

3. **claimableAssets** is zero, so the guard passes. The escrow closes, **SAFETY_MANAGER_ROLE** repays the outstanding debt out of pocket, and the receipt NFT is burned

4. Midas tops up the redeemer and approves the request the next day, paying the payment asset to the user wallet

5. A keeper calls **settleRequest**. **claimableRedeemRequest** returns the full gross shares, so the gate passes

6. **claimRedeem** succeeds and moves the assets to the manager

7. **ownerOf(requestState.receiptNftId)** reverts **ERC721NonexistentToken** and the whole transaction rolls back, including the claim

8. Every later retry fails at the same line. The proceeds stay in the user wallet with no in-protocol path out

---

Recommendation

First, stop the settlement path from reverting on a burned receipt. Use the non-reverting **_ownerOf** and skip the set removal when the token no longer exists.

```
        s_requestIdToClaimableAssets[_requestId] += redeemedAssets;
-
EnumerableSet.remove(s_pendingRequestIdsByUser[ownerOf(requestState.receiptN
ftId)], _requestId);
+       address receiptOwner = _ownerOf(requestState.receiptNftId);
+       if (receiptOwner != address(0)) {
+           EnumerableSet.remove(s_pendingRequestIdsByUser[receiptOwner],
_requestId);
+       }
```

That alone unbricks the claim but leaves the recovered assets sitting in the manager with no owner to pay, so it is not sufficient on its own.

Second, record the safety close so the recovered assets have a destination. Mark the request closed when **safetyCloseUnwind** runs, and add a manager entry point that sweeps a late settlement to whoever funded the close.

```
+    mapping(bytes32 requestId => address safetyCloser) internal
s_requestIdToSafetyCloser;
+
    function safetyCloseUnwind(bytes32 _requestId) external {
        _checkRoleMsgSender(SAFETY_MANAGER_ROLE);
        // ...
+       s_requestIdToSafetyCloser[_requestId] = msg.sender;
        _burn(vaultState.receiptNftId);
```

The late settlement then pays the safety closer rather than being trapped. Without the second change the first one only moves the stranded value from the user wallet into the manager.

3S-M03

Missing handling of Midas-cancelled redemptions in **MidasAdapter** leads to permanent loss of user collateral

Id	3S--M03
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged with note: “An exceptional path; not to be expected in regular operations. Confirmed by Midas team.”

Description

MidasAdapter redeems a user's RWA collateral (Midas mTokens) back into cash while a leveraged position is being unwound. It moves the mTokens into the Midas vault through **MidasAdapter::requestRedeem**, records the request as **Requested**, and assumes Midas will either keep it **Pending** or mark it **Processed** and pay out.

Midas has a third outcome the adapter ignores: a Midas admin can call **rejectRequest**, setting the request to **Canceled**. This is documented, routine behaviour and Midas does not return the mTokens automatically. The adapter's state machine only knows **None**, **Requested**, and **Claimed**, so once a request is cancelled it is stuck: settlement in **MidasAdapter::claimRedeem** reverts forever because Midas is not **Processed**, and there is no cancel or rescue function anywhere.

```
// MidasAdapter.sol:161-163 — claimRedeem can never pass again once cancelled
if (midasStatus != IMidasRedemptionVault.RequestStatus.Processed) {
    revert MidasAdapter__MidasRequestNotProcessed(request.midasRequestId,
midasStatus);
}
```

The collateral is stranded and the redemption can never finish. There is no built-in recovery — the only escape is an emergency admin rewiring of the adapter via **setExtensions**, which requires new code. Otherwise the position is closed only by the protocol paying the outstanding debt through **safetyCloseUnwind**, recovering none of the collateral.

Steps to Reproduce

1. A user unwinds a position; the manager moves their mToken collateral into the Midas vault via **requestRedeem**, recording status **Requested**.

2. A Midas admin calls **rejectRequest**; Midas sets the request to **Canceled** and keeps the mTokens.

3. **claimRedeem** now reverts on every call, and the same request id can never be re-submitted.

4. Both **claimableRedeemRequest** and **pendingRedeemRequest** return zero, so settlement silently no-ops — the stuck state is indistinguishable from "not processed yet".

5. No built-in path recovers the collateral; short of an emergency admin rewiring via **setExtensions**, the position can only be closed by the protocol paying the debt itself.

Recommendation

Add a **Canceled** state and a recovery path for cancelled redemptions.

- enum RequestStatus { None, Requested, Claimed }
+ enum RequestStatus { None, Requested, Claimed, Canceled }

3S-M04

Missing payout floor in **MidasAdapter::claimRedeem** leads to permanently insolvent unwind escrows

Id	3S--M04
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged with note: “Midas payout and pending settlement token valuation use the same current NAV, so a payout below the outstanding debt implies a NAV drawdown, which makes the escrow liquidatable and allows the liquidation auction to resolve the shortfall.”

Description

MidasAdapter::claimableRedeemRequest is the gate the manager uses to decide whether a request can be settled. It reads the Midas record, discards the payout fields, and returns the gross amount stored at request time:

```
        (,, IMidasRedemptionVault.RequestStatus status,,,) =
_validatedMidasRequest(request);
        return status == _expectedStatus ? request.shares : 0;
```

request.shares is **cache.collateralAssetsToRedeem**, passed at **UnwindManager.sol:235**. The same value is written once into the escrow at **UnwindEscrow.sol:74** and never mutated. So the manager compares that value against itself:

```
        uint256 totalSharesForRequest = requestState.collateralAssetsToRedeem;
        ...
        uint256 claimableShares = s_adapter.claimableRedeemRequest(_requestId,
userWallet, _adapterData);
        if (claimableShares < totalSharesForRequest) {
```

```
        return;
    }
```

The gate passes the moment Midas reports **Processed**. It never asks whether the payout covers the debt.

claimRedeem then applies a single floor before an irreversible state change:

```
    uint256 assets = ((netShares * mTokenRate) / tokenOutRate) / i_assetScale;
    if (assets == 0) revert
MidasAdapter__ZeroAssetsClaimed(request.midasRequestId);
    request.assets = assets;
    request.status = RequestStatus.Claimed;
```

Only zero is rejected. One wei commits. The operator cannot supply a minimum, because both **claimRedeem** and **claimableRedeemRequest** reject any non-empty payload, at **MidasAdapter.sol:150** and **:308**.

Once the recorded claimable amount is non-zero and below the debt, the position is insolvent and every exit is closed. **finalizeUnwind** reverts **UnwindManagerExtension__InsufficientAssetsToRepayDebt** at **UnwindManagerExtension.sol:182-184** and **debtOutstanding** only grows from there. **settleRequest** short circuits on the recorded amount being non-zero at **UnwindManager.sol:458-460**. **safetyCloseUnwind** reverts for the same reason at **UnwindManagerExtension.sol:235-237**, so the valve is disabled by the partial success it exists to rescue. The auction cannot open either, because a Midas approval rate below the Euler mark leaves the escrow reading as healthy, **checkLiquidation** returns a zero maximum repay, and both **UnwindLiquidationAuction.sol:123** and **UnwindManagerExtension.sol:334-336** revert.

The combined erosion needed is **1 - (liquidationLTV - buffer) / 10000**, so 15% at a 9000 bps liquidation LTV with a 500 bps buffer. Interest accrual alone reaches that only over hundreds of days, so the driver is rate movement rather than queue length. Midas can supply that movement directly, because **RedemptionVault::approveRequest** passes **isSafe** as false, which skips **_requireVariationTolerance** entirely and writes an unbounded rate to storage.

The condition is already recognised outside the contracts. **app/src/lib/protocols/euler/liquidation.ts:117** returns an **impaired_settlement** state on **claimablePayout < requiredEscrowDebt**. **docs/plans/2026-05-08-liquidation-rfq-settlement-scenarios.md:257** records that the shortfall case needs an explicit product and contract decision. The contracts make none, and no test exercises it. **InsufficientAssetsToRepayDebt** appears nowhere under

contracts/test/, and **MockUnwindAdapter.sol:112-115** falls back to **assets = _shares** whenever **s_claimAssets** is unset, which no test overrides.

Steps to Reproduce

1. A user unwinds 100,000 mToken against 85,000 USDC of debt. The mToken marks at 1.00, so **requesterLtv** is 8500 bps and the gate at **UnwindManager.sol:365** passes against a 9000 bps liquidation LTV and a 500 bps buffer

2. The adapter stores **request.shares** as the full 100,000 gross and submits the Midas request

3. 30 days pass, inside the 1 to 90 day window documented at **docs/unwind.md:194**. Interest takes **debtOf(escrow)** to 85,701 USDC

4. Midas approves at an mToken rate of 0.855 and pays 85,500 USDC to the user wallet

5. The keeper sees a non-zero claimable and calls **settleRequest**. The gate compares 100,000 against 100,000 and passes

6. **claimRedeem** computes 85,500, which is non-zero, and flips the request to **Claimed**

7. The escrow is short 201 USDC. **finalizeUnwind** and **safetyCloseUnwind** both revert, and no auction round can open because the escrow still marks as healthy

8. The debt keeps accruing with no way to close the position

Recommendation

Let the operator pass a floor through the adapter payload instead of rejecting all payloads. **settleRequest** already takes **_adapterData** from the caller, so the operator supplies the escrow's live **debtOutstanding** at settle time with no storage change.

```
        _requireManager();
-       if (_data.length != 0) revert MidasAdapter__SettlementDataNotEmpty();
+       uint256 minAssets = _data.length == 0 ? 0 : abi.decode(_data, (uint256));
        Request storage request = _validatedRequest(_requestId, _userWallet);
        if (_shares != request.shares) revert
MidasAdapter__InvalidShares(request.shares, _shares);
        // ...
        uint256 assets = ((netShares * mTokenRate) / tokenOutRate) / i_assetScale;
        if (assets == 0) revert
MidasAdapter__ZeroAssetsClaimed(request.midasRequestId);
+       if (assets < minAssets) revert
MidasAdapter__InsufficientAssetsClaimed(assets, minAssets);
```

The gate needs the same floor so a short request reports as not claimable rather than reverting the settle call:

```
-       (,, IMidasRedemptionVault.RequestStatus status,,,) =
_validatedMidasRequest(request);
+       (,, IMidasRedemptionVault.RequestStatus status, uint256 netShares, uint256
mTokenRate, uint256 tokenOutRate) =
+           _validatedMidasRequest(request);
+       if (status == _expectedStatus && _minAssets != 0) {
+           uint256 assets = ((netShares * mTokenRate) / tokenOutRate) / i_assetScale;
+           if (assets < _minAssets) return 0;
+       }

        return status == _expectedStatus ? request.shares : 0;
```

_viewRequestShares takes **_minAssets** as a new parameter, and **claimableRedeemRequest** decodes it from **_data** instead of rejecting it. The keeper at **packages/cli/src/ops/rwa/unwind.ts:139** then passes **debtOutstanding** as the floor rather than settling on a non-zero claimable amount.

3S-N01

Suboptimal field ordering in the Request struct wastes a storage slot and raises gas on every redemption

Id	3S--N01
Classification	None
Category	Optimization
Status	Acknowledged

Description

The adapter tracks each redemption in the **Request** struct (**IMidasAdapter.sol:27-33**):

```
struct Request {
    address userWallet;   // 20 bytes
    uint256 shares;
    uint256 midasRequestId;
    uint256 assets;
    RequestStatus status; // 1 byte (uint8)
}
```

Solidity packs only fields that are declared next to each other, so the 20-byte **userWallet** and the 1-byte **status** end up in separate storage slots and the struct occupies five slots. Since the two together need only 21 of the 32 bytes in a slot, placing **status** right after **userWallet** packs them into one slot and shrinks the struct to four.

Fewer slots means fewer storage reads and writes. **MidasAdapter::requestRedeem** writes one cold slot less (~22k gas saved per request), and every read path — **claimRedeem**, the view functions, and **getRequest** — touches one slot less. This is a pure optimization; no behaviour changes.

Recommendation

Reorder the struct so **status** follows **userWallet**:

```
 struct Request {
     address userWallet;
+    RequestStatus status;
```

```
    uint256 shares;
    uint256 midasRequestId;
    uint256 assets;
-   RequestStatus status;
}
```

3S-N02

Re-validating live Midas state during claimRedeem can render already-delivered funds inaccessible

Id	3S--N02
Classification	None
Category	Suggestion
Status	Acknowledged

Description

Once a Midas redemption is processed, Midas transfers the output assets directly into the user's **UserWallet**. The protocol then collects them by calling **MidasAdapter::claimRedeem**, which pulls the assets from the wallet into the manager.

The concern is that **claimRedeem** does not simply move funds that have already arrived. Through **MidasAdapter::_validatedMidasRequest** it re-reads the live Midas request, reverts if that record is no longer well-formed, and recomputes the claim amount from Midas's stored **netShares**, **mTokenRate**, and **tokenOutRate**:

```
// MidasAdapter.sol:223-228
if (
    sender != _request.userWallet || tokenOut != address(i_asset) || netShares == 0
        || netShares > _request.shares || mTokenRate == 0 || tokenOutRate == 0
) {
    revert MidasAdapter__InvalidMidasRequest(_request.midasRequestId);
}
```

So even when the assets are already sitting in the wallet, access to them stays coupled to external Midas state. If that state ever became invalid or unavailable, **claimRedeem** would revert and the already-delivered funds would be stuck in the wallet, with no other way out (only the bound adapter can move them, and it has no sweep).

This is a theoretical design observation, not an exploitable bug. It is raised because the protocol keeps no fallback for such a scenario.

Recommendation

Consider providing a fallback path so that, should the Midas record ever become invalid or unavailable, the manager can still recover assets that have already arrived in the wallet, without depending on live Midas validation.